

LLM-Assisted Long-Horizon Predictive Modeling of High-Dimensional Chaotic Systems

A Case Study on Lorenz-63 and Lorenz-96

Xingtao Zhong^{*} Yuxuan Huang^{*} Zhiqi Hao

^{*}Co-first authors / equal contribution

Student IDs: 25120617 25120619 25120621

Course: Introduction to Artificial Intelligence

June 10, 2026

Abstract

This paper studies long-horizon multi-step prediction for chaotic dynamical systems. With the assistance of large language models, we first use the Lorenz-63 system to show that ordinary supervised learning models can fit short-term state transitions but fail to maintain stable recursive rollout. We then construct a 10-dimensional Lorenz-96 dataset and compare several models, including Baseline LSTM, PINN LSTM, Transformer, Hybrid LSTM, and Ultimate Hybrid. The main evaluation metric is the final cumulative RMSE over a 500-step rollout. The results show that one-step accuracy alone does not guarantee long-term stability. The proposed Ultimate Hybrid model, which combines an imperfect RK4 physical solver, Transformer-based residual learning, PINN loss, and refined scheduled sampling, achieves the lowest final cumulative RMSE of 12.80. The study suggests that, for high-dimensional chaotic prediction, machine learning is more effective as a residual corrector for physical models than as a fully black-box replacement.

Keywords: large language model; Lorenz-96; high-dimensional chaotic system; long-horizon prediction; recursive rollout; Transformer; PINN; hybrid correction

Contents

1	Introduction	4
2	Dataset Construction and Exploratory Analysis	4
2.1	Data Source and Feature Description	4
2.2	Correlation Heatmap	5
2.3	Preprocessing	5
3	Lorenz-63: Short-Term Learnability and Long-Term Limitation	6
3.1	Lorenz-63 Equation and Experimental Purpose	6
3.2	Short-Term Prediction and Rollout Divergence	6
4	Lorenz-96 and Multi-Step Prediction Definition	8
4.1	Lorenz-96 Dynamics	8
4.2	Window-Based Sequence Prediction	8
4.3	Cumulative RMSE	9
5	Model Design: From Baseline LSTM to Ultimate Hybrid	9
5.1	Baseline LSTM and State-Increment Learning	9
5.2	Scheduled Sampling	9
5.3	PINN Physical Constraint	10
5.4	Transformer for Historical Windows	10
5.5	Imperfect Physical Solver and Residual Correction	11
5.6	Ultimate Hybrid Architecture	11
6	Results and Analysis	11
6.1	Stage-Wise LSTM Optimization	11
6.2	Window-Length Ablation	12
6.3	Advanced Architecture Comparison	13
6.4	Prediction Scatter Analysis	13
6.5	Component Contribution	14
6.6	Why One-Step Accuracy Does Not Equal Long-Term Stability	14
7	Discussion	15
7.1	Division of Labor Between Physics and Deep Learning	15
7.2	Limitations of Different Modules	15
7.3	Experimental Limitations	15

8	Conclusion	16
9	Reflection on AI Collaboration	16
A	LLM Interaction and Prompt Engineering Log	16
A.1	Problem Definition and Architecture Design	16
A.2	Core Debugging Record	17
B	Reproducibility	17

1 Introduction

The main difficulty of chaotic-system prediction is not only the one-step prediction error, but also the amplification of small errors during long recursive rollout. For deterministic nonlinear systems such as the Lorenz family, a tiny deviation in the initial or predicted state may lead to a rapidly diverging trajectory. Therefore, a low one-step RMSE on a test set does not necessarily mean that the model can stably predict hundreds of future steps.

This paper focuses on the following question: how can we design a predictive model that combines physical structure, sequence memory, and long-horizon stability for high-dimensional chaotic systems? We first use the Lorenz-63 system as an intuitive baseline case. Its three-dimensional structure makes it suitable for demonstrating that short-term local state transitions are learnable, while long-horizon recursive rollout remains difficult. The central task of this paper is then extended to the 10-dimensional Lorenz-96 system, whose higher dimensionality and periodic coupling make long-term prediction more challenging.

The main contributions of this work are:

1. We use Lorenz-63 to verify the phenomenon that short-term prediction is learnable but long-term rollout is limited by chaotic error amplification.
2. We build a recursive rollout evaluation framework for the Lorenz-96 system and use cumulative RMSE as the primary metric.
3. We compare Residual LSTM, PINN LSTM, Transformer, Hybrid LSTM, and Ultimate Hybrid models under the same long-horizon setting.
4. We propose an Ultimate Hybrid architecture that combines an imperfect physical solver, Transformer residual learning, PINN constraints, and refined scheduled sampling.
5. We analyze the contribution and limitation of each component based on reproducible CSV outputs and generated figures.

2 Dataset Construction and Exploratory Analysis

2.1 Data Source and Feature Description

The data used in this project are generated by numerical integration of dynamical-system equations rather than downloaded from a static external dataset. The Lorenz-63 part contains three state variables, x , y , and z , and forms a short-term supervised-learning task such as predicting $x(t + k\Delta t)$. The Lorenz-96 part contains $N = 10$ coupled state

variables, $x_1, \dots, x_{10}$, and the main task is to predict the next full state vector from a historical window.

From a machine-learning perspective, each sample corresponds to either a single time slice or a historical time window. The input features describe the current position and local evolution trend in phase space, while the target variable describes the next state or state increment. All experimental data are generated by Python scripts and saved in `results/*.csv`, making the workflow reproducible and traceable.

2.2 Correlation Heatmap

Figure 1 shows the correlation heatmap for the Lorenz-63 supervised-learning dataset. The heatmap indicates that neighboring states and future targets are statistically related, which explains why short-term local prediction is feasible. However, such local correlation does not imply long-term predictability, because recursive rollout repeatedly feeds small model errors back into the system.

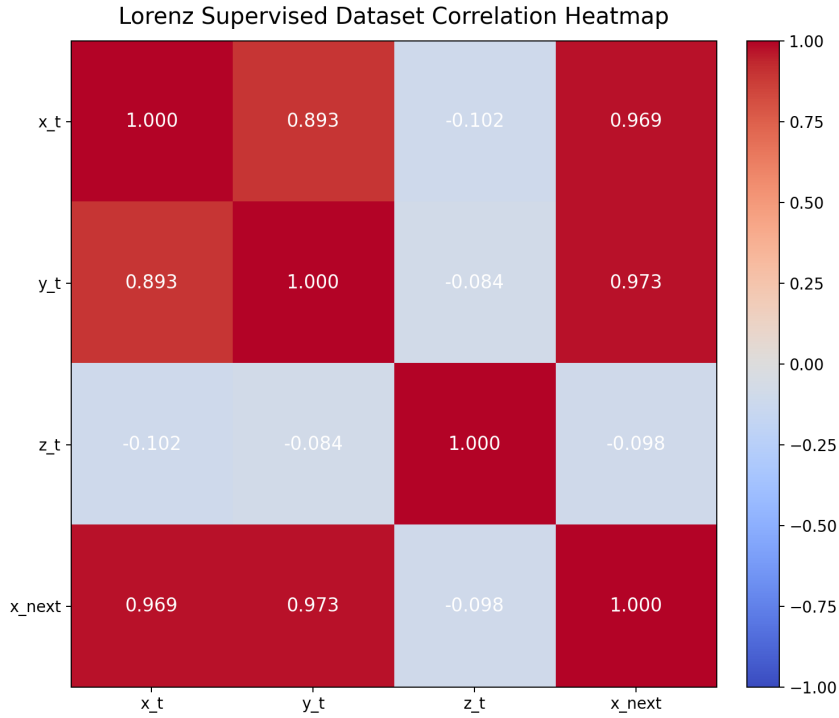


Figure 1 Correlation heatmap of the Lorenz-63 supervised-learning dataset. The heatmap checks linear feature relations and supports the claim that short-term local states are learnable.

2.3 Preprocessing

The preprocessing pipeline includes missing-value checking, chronological train-test splitting, and standardization. Since the data are generated by numerical integration, no missing values are expected, but the output files are still checked. The train and test sets are split by time order instead of random shuffling to avoid future information leakage.

For models based on gradient descent, including MLP, LSTM, and Transformer, `StandardScaler` is used to place different variables on comparable scales. This improves optimization stability and prevents dimensions with larger numerical ranges from dominating the loss. Tree-based models do not strictly require standardization, but a unified preprocessing pipeline makes the experimental procedure easier to verify.

3 Lorenz-63: Short-Term Learnability and Long-Term Limitation

3.1 Lorenz-63 Equation and Experimental Purpose

The classical Lorenz-63 system is defined by three coupled ordinary differential equations:

$$\frac{dx}{dt} = \sigma(y - x), \quad (1)$$

$$\frac{dy}{dt} = x(\rho - z) - y, \quad (2)$$

$$\frac{dz}{dt} = xy - \beta z. \quad (3)$$

We use the standard parameter setting

$$\sigma = 10, \quad \rho = 28, \quad \beta = \frac{8}{3}. \quad (4)$$

The trajectory is generated using `scipy.integrate.solve_ivp`, with initial state $(1, 1, 1)$, simulation interval $[0, 50]$, and 10000 sampled points. Figure 2 shows the generated Lorenz attractor.

3.2 Short-Term Prediction and Rollout Divergence

Table 1 reports the test-set results for the original short-term task, predicting $x(t + 10\Delta t)$. Random Forest and MLP clearly outperform Linear Regression, showing that local nonlinear mappings can be learned by supervised models.

Table 1 Short-term single-variable prediction results for Lorenz-63. Data source: `results/model_metrics.csv`.

Model	RMSE	MAE	R^2
Linear Regression	0.5457	0.4437	0.9952
Random Forest	0.0805	0.0507	0.9999
MLP	0.0757	0.0599	0.9999

Lorenz-63 Three-Dimensional Attractor

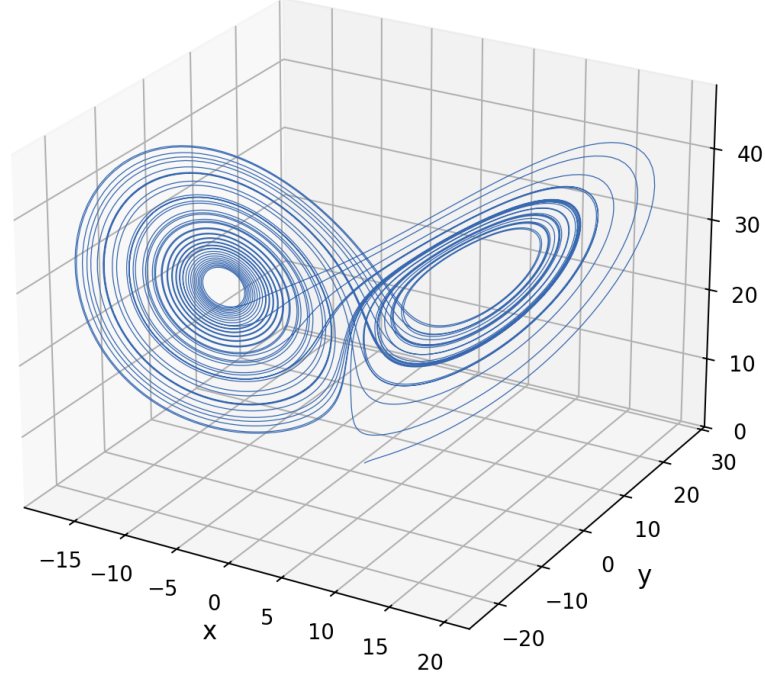


Figure 2 Three-dimensional Lorenz-63 attractor. This system is used to illustrate short-term fitting and long-term divergence in chaotic prediction.

However, when the prediction horizon is expanded to $k = 500$, the state RMSE rises to 10.9789 for Random Forest and 14.0980 for MLP. In a stricter 500-step recursive rollout setting, the model only receives the true state at the initial time and then repeatedly uses its own prediction as input. The final cumulative RMSE is shown in Table 2.

Table 2 500-step recursive rollout results for Lorenz-63 baseline models. Data source: `results/rollout_results.csv`.

Model	Final Cumulative State RMSE
Linear Regression	20.2244
Random Forest	14.5356
MLP	19.8142

This result motivates the Lorenz-96 experiments: the real challenge is not one-step fitting, but long-horizon stability.

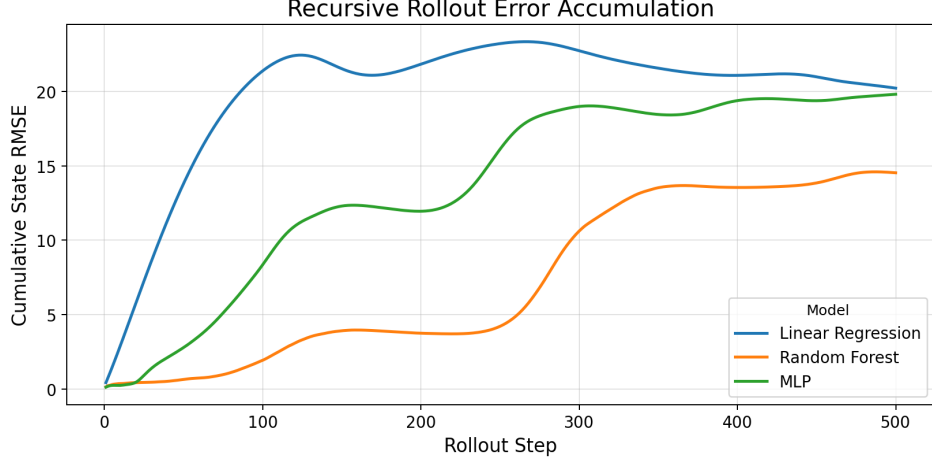


Figure 3 Cumulative error of Lorenz-63 baseline models during recursive rollout. Short-term errors are repeatedly fed into the next step and gradually become long-term trajectory deviations.

4 Lorenz-96 and Multi-Step Prediction Definition

4.1 Lorenz-96 Dynamics

Lorenz-96 is a common benchmark system for high-dimensional chaotic prediction. This paper uses the $N = 10$ dimensional system:

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F, \quad i = 1, \dots, N. \quad (5)$$

Periodic boundary conditions are applied:

$$x_{i+N} = x_i. \quad (6)$$

The true forcing parameter is set to

$$F_{\text{true}} = 8.0, \quad (7)$$

which places the system in a chaotic regime. The simulation interval is $[0, 100]$, with 10000 sampled points. The initial state is a small perturbation around F :

$$\mathbf{x}_0 = (F + 0.01, F, \dots, F). \quad (8)$$

Training and test sets are split chronologically into the first 80% and last 20%.

4.2 Window-Based Sequence Prediction

For Lorenz-96, the model input is a historical window of length w :

$$\mathbf{X}_t = (\mathbf{x}_{t-w+1}, \mathbf{x}_{t-w+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{w \times N}. \quad (9)$$

The one-step prediction task is

$$\hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{X}_t). \quad (10)$$

During evaluation, recursive rollout is used:

$$\hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{x}_{t-w+1}, \dots, \mathbf{x}_t), \quad (11)$$

$$\hat{\mathbf{x}}_{t+2} = f_{\theta}(\mathbf{x}_{t-w+2}, \dots, \mathbf{x}_t, \hat{\mathbf{x}}_{t+1}), \quad (12)$$

$$\vdots$$

$$\hat{\mathbf{x}}_{t+m} = f_{\theta}(\hat{\mathbf{x}}_{t+m-w}, \dots, \hat{\mathbf{x}}_{t+m-1}). \quad (13)$$

This evaluation setting is closer to real long-term prediction and more clearly exposes stability problems.

4.3 Cumulative RMSE

We use the final cumulative RMSE over a 500-step rollout:

$$\text{CRMSE}(T) = \sqrt{\frac{1}{T} \sum_{m=1}^T \|\mathbf{x}_{t+m} - \hat{\mathbf{x}}_{t+m}\|_2^2}. \quad (14)$$

Compared with one-step RMSE, CRMSE(T) reflects both per-step error and error accumulation under recursive input. All main Lorenz-96 results use $T = 500$.

5 Model Design: From Baseline LSTM to Ultimate Hybrid

5.1 Baseline LSTM and State-Increment Learning

The natural baseline for sequence prediction is LSTM. Direct state prediction can be written as

$$\hat{\mathbf{x}}_{t+1} = f_{\theta}^{\text{LSTM}}(\mathbf{X}_t). \quad (15)$$

However, for continuous dynamical systems, the change between adjacent time steps is often easier to learn than the absolute state. Therefore, the LSTM is converted into a residual form:

$$\Delta \hat{\mathbf{x}}_{t+1} = g_{\theta}^{\text{LSTM}}(\mathbf{X}_t), \quad (16)$$

$$\hat{\mathbf{x}}_{t+1} = \mathbf{x}_t + \Delta \hat{\mathbf{x}}_{t+1}. \quad (17)$$

This design makes the model closer to a local numerical integrator.

5.2 Scheduled Sampling

In one-step training, the model always sees a true historical window. In rollout, it gradually sees its own predicted states. This difference is known as exposure bias or train-test mismatch. Scheduled sampling reduces the mismatch by using model predictions with increasing probability during multi-step training.

Let p_e denote the teacher-forcing probability at epoch e . A simple linear decay is

$$p_e = \max\left(0, 1 - \frac{e}{0.8E}\right), \quad (18)$$

where E is the total number of epochs. In the Ultimate Hybrid model, a smoother inverse-sigmoid schedule is used:

$$p_e = \frac{k}{k + \exp(e/(E/10))}. \quad (19)$$

The model is unrolled for $M = 5$ steps during training:

$$\mathcal{L}_{multi} = \frac{1}{M} \sum_{m=1}^M \text{MSE}(\hat{\mathbf{x}}_{t+m}, \mathbf{x}_{t+m}). \quad (20)$$

5.3 PINN Physical Constraint

Pure data-driven models may produce updates that violate the underlying dynamics. We introduce a physics-informed neural network (PINN) loss [4]. The Lorenz-96 vector field is denoted as

$$\mathbf{f}_{L96}(\mathbf{x}; F_{\text{true}}). \quad (21)$$

The finite-difference derivative implied by the model prediction is

$$\frac{\hat{\mathbf{x}}_{t+1} - \mathbf{x}_t}{\Delta t}. \quad (22)$$

The physical loss is defined as

$$\mathcal{L}_{phys} = \left\| \frac{\hat{\mathbf{x}}_{t+1} - \mathbf{x}_t}{\Delta t} - \mathbf{f}_{L96}(\mathbf{x}_t; F_{\text{true}}) \right\|_2^2. \quad (23)$$

The total loss is

$$\mathcal{L} = \mathcal{L}_{data} + \lambda \mathcal{L}_{phys}, \quad (24)$$

with $\lambda = 0.1$ in the implementation.

5.4 Transformer for Historical Windows

LSTM compresses historical information into recurrent hidden states, while Transformer uses self-attention to directly model relations among time steps [3]. The Transformer model embeds each 10-dimensional state into a 64-dimensional space, adds positional encoding, and processes the sequence using a two-layer Transformer encoder:

$$\mathbf{Z}_t = \text{PE}(\text{Linear}(\mathbf{X}_t)), \quad (25)$$

$$\mathbf{H}_t = \text{TransformerEncoder}(\mathbf{Z}_t), \quad (26)$$

$$\Delta \hat{\mathbf{x}}_{t+1} = W \mathbf{H}_{t,-1} + b. \quad (27)$$

This structure allows the model to use local oscillations and short-term evolution trends inside the historical window.

5.5 Imperfect Physical Solver and Residual Correction

In many scientific problems, an approximate physical model is available. We simulate this situation by constructing an imperfect Lorenz-96 solver:

$$F_{\text{imp}} = 8.5, \quad F_{\text{true}} = 8.0. \quad (28)$$

Let $\Phi_{\Delta t}^{F_{\text{imp}}}$ denote a one-step RK4 solver using F_{imp} :

$$\mathbf{x}_{t+1}^{\text{phys}} = \Phi_{\Delta t}^{F_{\text{imp}}}(\mathbf{x}_t). \quad (29)$$

The hybrid model predicts the residual of this imperfect physical prediction:

$$\Delta \hat{\mathbf{x}}_{t+1}^{\text{res}} = g_{\theta}(\mathbf{X}_t), \quad (30)$$

$$\hat{\mathbf{x}}_{t+1} = \mathbf{x}_{t+1}^{\text{phys}} + \Delta \hat{\mathbf{x}}_{t+1}^{\text{res}}. \quad (31)$$

Thus, the physical solver provides the main evolution direction, while the neural network only corrects systematic bias and approximation error.

5.6 Ultimate Hybrid Architecture

The Ultimate Hybrid model combines four components:

1. an imperfect RK4 physical solver with $F_{\text{imp}} = 8.5$;
2. a Transformer residual model that extracts features from historical windows;
3. a PINN loss that constrains the predicted update direction;
4. refined scheduled sampling with 5-step unrolled training.

Mathematically, its one-step prediction is

$$\hat{\mathbf{x}}_{t+1} = \Phi_{\Delta t}^{F_{\text{imp}}}(\mathbf{x}_t) + g_{\theta}^{\text{Transformer}}(\mathbf{x}_{t-w+1}, \dots, \mathbf{x}_t). \quad (32)$$

The training objective is

$$\mathcal{L}_{UH} = \frac{1}{M} \sum_{m=1}^M \left[\text{MSE}(\hat{\mathbf{x}}_{t+m}, \mathbf{x}_{t+m}) + \lambda \left\| \frac{\hat{\mathbf{x}}_{t+m} - \tilde{\mathbf{x}}_{t+m-1}}{\Delta t} - \mathbf{f}_{L96}(\tilde{\mathbf{x}}_{t+m-1}; F_{\text{true}}) \right\|_2^2 \right], \quad (33)$$

where $\tilde{\mathbf{x}}_{t+m-1}$ denotes the input state after scheduled sampling.

6 Results and Analysis

6.1 Stage-Wise LSTM Optimization

Table 3 summarizes representative results from the Lorenz-96 LSTM optimization process. Baseline LSTM achieves a reasonable one-step score but still accumulates significant long-term error. Residual learning, LayerNorm, and gradient clipping improve

Phase 1. However, the simple scheduled-sampling version in Phase 2 improves one-step RMSE but worsens 500-step rollout RMSE. This shows that multi-step training must be carefully designed for chaotic systems.

Table 3 Stage-wise Lorenz-96 LSTM optimization results.

Model Stage	One-step RMSE	One-step R^2	Final 500-step RMSE
Baseline LSTM	0.7360	0.9531	13.9662
Phase 1: residual + LayerNorm + clipping	0.0417	0.9999	13.1749
Phase 2: multi-step loss + scheduled sampling	0.0356	0.9999	14.1880

6.2 Window-Length Ablation

The historical window length w determines how much past information is visible to the model. Too short a window may miss useful phase-space information, while too long a window may introduce redundant chaotic noise. Table 4 and Figure 4 show the ablation results.

Table 4 Lorenz-96 LSTM window-length ablation. Data source: results/lstm_phase3_summary.csv.

Window Length w	Final 500-step RMSE
10	15.7545
20	16.5752
50	111.0108



Figure 4 Lorenz-96 window-length ablation. The long window $w = 50$ severely worsens rollout stability, showing that more history is not always better in chaotic sequences.

6.3 Advanced Architecture Comparison

Table 5 compares advanced models on the Lorenz-96 500-step rollout task. Baseline LSTM performs worst, with a final cumulative RMSE of 148.79. PINN, Transformer, Refined SS, and Hybrid LSTM all significantly reduce long-term error. Ultimate Hybrid achieves the best result, with a final cumulative RMSE of 12.80.

Table 5 Advanced Lorenz-96 (10D) architecture comparison under 500-step rollout. Data source: `results/advanced/summary.csv`.

Model Architecture	Final Cumulative State RMSE
Baseline LSTM	148.79
PINN LSTM	20.18
Refined SS LSTM	16.79
Transformer	16.34
Hybrid LSTM	15.84
Ultimate Hybrid	12.80

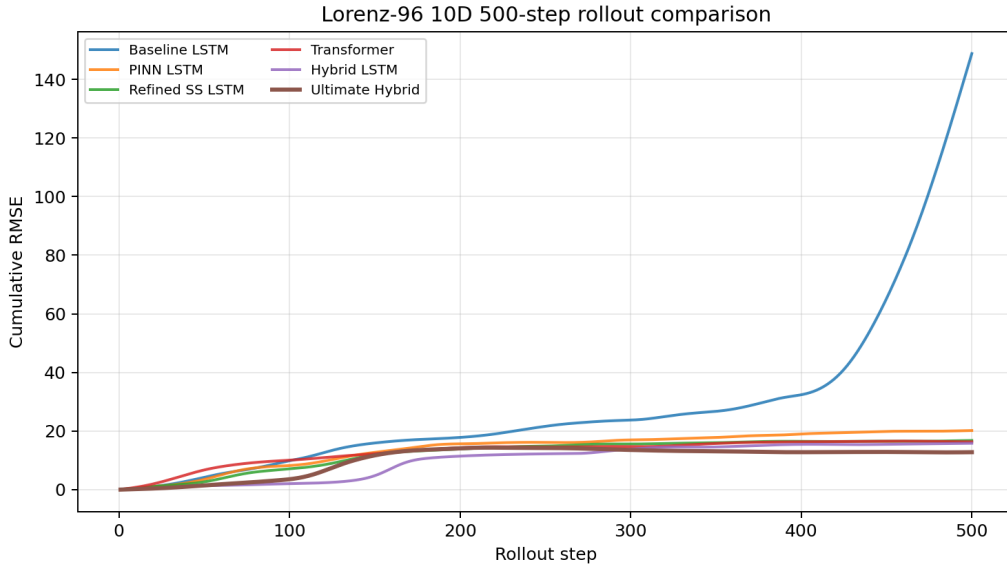


Figure 5 Cumulative RMSE curves of advanced Lorenz-96 models under 500-step rollout. Ultimate Hybrid maintains lower error across the test window and achieves the best final RMSE.

6.4 Prediction Scatter Analysis

Following the course requirement for a “true value vs. predicted value” scatter plot, Figure 6 shows the prediction scatter of Random Forest on the Lorenz-63 short-term task. Most points lie close to the diagonal line, indicating good local short-term fitting. In extreme regions where the absolute value of x is large, the scatter becomes slightly more dispersed, suggesting that highly nonlinear states are more difficult to predict.

This scatter plot mainly reflects one-step or short-horizon fitting quality. It should not be interpreted as evidence of long-term stability. The Lorenz-96 rollout results show that even if the short-term scatter is close to the diagonal, recursive prediction can still diverge due to chaotic sensitivity.

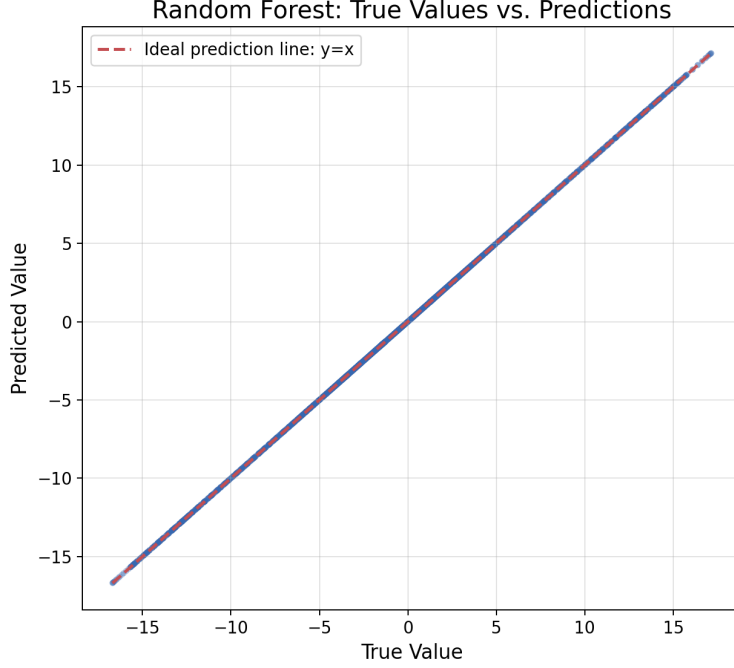


Figure 6 True value vs. predicted value scatter plot for the Lorenz-63 short-term prediction task. Most points are close to the diagonal, but extreme states show larger deviations.

6.5 Component Contribution

PINN LSTM greatly reduces error compared with Baseline LSTM, showing that physical derivative constraints can suppress nonphysical extrapolation. Transformer outperforms Refined SS LSTM, suggesting that self-attention is useful for extracting local evolution patterns from historical windows. Hybrid LSTM further improves the result, showing that even an imperfect physical solver can provide a valuable dynamical basis. Ultimate Hybrid combines these advantages: the physical solver provides a basic evolution direction, Transformer models residuals, PINN constrains update directions, and scheduled sampling brings the training environment closer to rollout.

6.6 Why One-Step Accuracy Does Not Equal Long-Term Stability

The results repeatedly show that one-step RMSE and rollout RMSE are not monotonically related. The reasons include:

1. one-step evaluation uses true historical windows, while rollout uses predicted historical windows;
2. high-dimensional chaotic systems amplify small per-step errors;
3. once predictions leave the neighborhood of the true attractor, later input distributions differ from training data;
4. a low one-step MSE may reflect local average fitting rather than preservation of dynamical structure.

Therefore, long-horizon rollout stability should be treated as the central evaluation criterion for high-dimensional chaotic prediction.

7 Discussion

7.1 Division of Labor Between Physics and Deep Learning

The Ultimate Hybrid design reflects a key idea in AI for Science: machine learning does not have to replace physical models; instead, it can cooperate with them. For Lorenz-96, the imperfect RK4 solver already contains the main dynamical structure, while Transformer only needs to learn the residual caused by parameter bias and model error. This decomposition is more stable than learning the entire state transition from data.

7.2 Limitations of Different Modules

Residual learning reduces the learning difficulty but does not guarantee long-term stability. Scheduled sampling can reduce train-test mismatch, but an inappropriate schedule may expose the model to noisy predictions too early. PINN loss constrains the update direction but requires a proper loss weight. Transformer improves historical-window modeling but increases training cost. Hybrid physics depends on the physical model being structurally reasonable; if the physical model is seriously wrong, residual learning may also fail.

7.3 Experimental Limitations

This project is still limited by the scale of a course assignment. Lorenz-96 is tested only with $N = 10$ and $F = 8.0$. The number of training epochs is fixed at 20, and large-scale hyperparameter search is not performed. The results mainly come from a single random seed. In future work, more systematic ablation studies could be conducted, such as Transformer + PINN, Transformer + Hybrid, and Hybrid + PINN combinations.

8 Conclusion

Starting from Lorenz-63, this paper shows that short-term local state transitions can be learned by supervised models, but long-term recursive rollout is limited by chaotic error amplification. The study then focuses on the 10-dimensional Lorenz-96 system and compares multiple deep-learning architectures under 500-step rollout.

The experiments demonstrate that one-step prediction accuracy is not sufficient to represent long-term prediction ability. Residual LSTM, PINN, Transformer, hybrid physics, and scheduled sampling improve rollout stability from different perspectives. The final Ultimate Hybrid model, which integrates an imperfect physical solver, Transformer residual modeling, PINN loss, and refined scheduled sampling, achieves the lowest final cumulative RMSE of 12.80.

Overall, the main conclusion is that high-dimensional chaotic prediction should not rely only on black-box one-step supervised learning. A more effective route is to use physical models as the dynamical backbone, apply deep learning to residual correction, and improve rollout stability through multi-step training and physical constraints.

9 Reflection on AI Collaboration

In this project, LLMs were mainly used for topic decomposition, experimental-route discussion, debugging suggestions, result organization, and report restructuring. They helped us realize that “good one-step prediction” is not a sufficient research conclusion, and pushed the project from Lorenz-63 short-term supervised learning to Lorenz-96 long-horizon architecture design.

At the same time, LLM-generated explanations must be verified by code and data. All numerical values in this report come from actual CSV outputs, and the formulas and model descriptions are checked against Python implementations. In this project, LLMs are collaboration tools rather than sources of experimental conclusions.

A LLM Interaction and Prompt Engineering Log

A.1 Problem Definition and Architecture Design

At the early stage, we asked the LLM the following core question: “We can already use MLP and Random Forest to make short-term predictions for Lorenz-63, but long-term rollout diverges quickly. Please help us extend this course project into a machine-learning modeling problem that better reflects AI for Science, and suggest a reasonable model comparison route.” The LLM suggested shifting the evaluation focus from one-step RMSE to recursive rollout and introducing Lorenz-96 as a high-dimensional chaotic system.

Based on this suggestion, we defined the research problem as predicting future Lorenz-96 states from historical windows while maintaining 500-step rollout stability. The LLM also suggested organizing the project around a progression from pure data-driven models to physics-constrained models and finally to physics-residual hybrid models. This became the basis for the Baseline LSTM, PINN LSTM, Transformer, Hybrid LSTM, and Ultimate Hybrid comparison.

A.2 Core Debugging Record

A typical debugging case occurred when extending LSTM from one-step training to multi-step scheduled sampling. The original code trained only on true historical windows, while rollout repeatedly appended predicted states back into the window. We sent tensor shapes, window-update code, and error messages to the LLM and asked how to concatenate predicted states into a tensor of shape `[batch, window, dim]`. The LLM pointed out that the oldest state should be removed along the time dimension and that `pred.unsqueeze(1)` should be concatenated to the end of the window.

The key lesson was that the LLM did not know our data dimensions unless we explicitly provided context such as `X.shape`, `pred.shape`, window length, state dimension, and the full error message. Therefore, later interactions treated LLM code as suggestions to be verified rather than final answers.

B Reproducibility

The main experiment files include:

- `lorenz_chaos_prediction.ipynb`: Lorenz-63 supervised learning and hybrid correction;
- `lorenz96_lstm.py`, `lorenz96_lstm_phase2.py`, and `lorenz96_lstm_phase3.py`: stage-wise Lorenz-96 LSTM optimization;
- `lorenz96_advanced_experiments.py`: comparison among PINN, Transformer, Hybrid LSTM, and Ultimate Hybrid.

Basic results are saved in `results/*.csv`, and advanced model results are saved in `results/advanced/*.csv`. The main comparison table in this report is derived from `results/advanced/summary.csv`, and rollout curves are generated from the corresponding `*_rollout.csv` files.

References

- [1] Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2), 130–141.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.
- [4] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.
- [5] Pedregosa, F., Varoquaux, G., Gramfort, M., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, M., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Virtanen, P., Gommers, R., Oliphant, M., Haberland, M., Reddy, T., Cournapeau, D., Peterson, P., Weckesser, W., Bright, J., van der Walt, S., Brett, M., Wilson, W., Millman, K., Mayorov, N., Nelson, A., Jones, E., Kern, R., Larson, E., Carey, C., Polat, I., Feng, Y., Moore, E., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E., Harris, C., Archibald, A., Ribeiro, A., Pedregosa, F., & van Mulbregt, P. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.